

Process Cooperation

Cooperation:

Cooperation means that processes work together on the same shared data, unlike communication where the data is shared between processes and accessed separately by each process.

The Problem: if 2 different processes run at the same time on the same shared data this can be dangerous in some situations if we don't implement control over it. Somethings like **race conditions, data corruption and deadlocks** then takes place.

Processes cooperation is implemented in both Unix system V and Posix System.

Access Control: to safely share processes between processes by implementing some controls to make sure they run safely together by restricting the access to critical sections.

Some access controls: signal – wait • lock – unlock • Semaphore • Mutex • Monitor

Semaphore:

The semaphore is an integer that allows N number of processes to access their critical code at the same time.

We have 2 symbols P and V where.

P which means proberien or try uses the function wait(). When this function is sent the Semaphore is checked if its greater than zero, if its greater than zero we decrement it by one and give a process the approval to execute its critical code. If its equal zero the process is then blocked.

V which mean vorhangen uses the Signal() function. Once a process that is executing a critical code finishes, it gives away a key and increments the S value.

Its very important to know that the S value is executed indivisibly which means only one process can increment or decrement its value at a time.

The main advantage of Semaphores is that we can make more than one process execute their critical code.

Producer – Consumer Example I:

Producers: the processes that generates data

Consumers: the processes that actually consumes this data.

Buffer: bounded memory to store produced data from the producers.

The thing is that processes works at different speeds, some consumers reads data faster than their producers and the opposite, so this data needs to be stored **temporarily** until its used. Minimizing waiting times for consumer, and providing space for data which is are not consumed in time

When the buffer is full the producer should wait and when the buffer is empty the consumer should wait.

For producers and consumers we need mutex to prevent data corruption.

Producer – Consumer Example II:

in this case we have 3 semaphore values:

Empty counts the empty spaces and keeps decreasing each time the producers produces data into the buffer and once it hits zero the producer stops writing data. Whenever the consumer calls V (reads data) the value is increased. Value is decreased by the producer P value.

Filled this counts how many buffer spaces are filled it keeps decreasing until it reaches zero when it reaches zero then the consumer is blocked. Its increased by the P value and decreased by the P value

Mutex ensures only one process executes their critical code

C-System Calls for Semaphores in Posix:

Creates a **new semaphore in memory** (accessible only by related processes, e.g., threads in same program). You can set its **initial value**.
sem_init()

Creates a **semaphore with a name** in the filesystem
sem_open() Named (/dev/shm/sem.<name>). Any process that knows the name can access it. You can also set **initial value**.

sem_post()	V operation / increment → signals that the resource is free or adds to the semaphore count
sem_wait()	P operation / decrement → tries to enter critical section. Blocks if semaphore value ≤ 0
sem_trywait()	Same as sem_wait(), but doesn't block ; returns immediately if not available
sem_timedwait()	Same as sem_wait(), but blocks with a timeout
sem_getvalue()	Returns the current value of the semaphore
sem_destroy()	Deletes an unnamed semaphore from memory
sem_close()	Closes a named semaphore descriptor (does not remove it from the system)
sem_unlink()	Removes a named semaphore from the system (/dev/shm/sem.<name>)

Producer – Consumer C-Code Example with Posix API:

1. Producer program

- Waits for an empty slot (sem_wait(empty_sem))
- Locks the buffer (sem_wait(mutex_sem))
- Writes data into pc_shm
- Unlocks buffer (sem_post(mutex_sem))
- Signals a filled slot (sem_post(full_sem))

2. Consumer program

- Waits for a filled slot (sem_wait(full_sem))
- Locks the buffer (sem_wait(mutex_sem))
- Reads data from pc_shm
- Unlocks buffer (sem_post(mutex_sem))
- Signals an empty slot (sem_post(empty_sem))

3. Files in /dev/shm/

After running the programs, Linux creates:

File	Purpose
pc_shm	Shared memory buffer for producer/consumer data
sem.empty_sem	Semaphore counting empty buffer slots
sem.full_sem	Semaphore counting filled buffer slots
sem.mutex_sem	Mutex semaphore for mutual exclusion

- These files **persist in /dev/shm/** until explicitly removed.

Mutex Vs Binary Semaphore:

Mutex works exactly the same as 0/1 binary semaphore

Locked = 1, some process already has the lock and working on it

Unlocked = 0, the lock is released and available

Function	Purpose
pthread_mutex_init()	Initialize a mutex
pthread_mutex_lock()	Lock the mutex (block if already locked)
pthread_mutex_trylock()	Try to lock without blocking; fails if already locked
pthread_mutex_timedlock()	Try to lock, block only up to a timeout
pthread_mutex_unlock()	Unlock the mutex
pthread_mutex_destroy()	Delete the mutex when done

Monitor:

A high level abstract method that used for synchronization that enforces mutex, you don't have to write lock and unlock. It encapsulates data automatically. And only the proper functions with proper access rights can access this data.

Only one process can execute a monitor at a time, automatic mutex.

In Java, the JVM (runtime) enforces monitor rules.

Advantages: less error prone, easier to use.

Example for monitoring in java with methods: Synchronized, Wait, notify, notifyAll

No monitor concept for Pascal, C, C++ or PHP.